

Chapter - 03

8. Queuing Diagram Representation of Process Scheduling

The queuing diagram represents how processes move among different queues during execution in an operating system.

Initially, a new process enters the **ready queue** where it waits for CPU allocation. When the scheduler selects the process, it is dispatched to the CPU for execution.

While executing, several events may occur:

1. The process may request an I/O operation. In this case, it is moved to the **I/O wait queue** until the I/O operation is completed.
2. The process may create a child process and wait for the child to terminate. Then it is placed in the **child termination wait queue**.
3. The process may wait for an interrupt and be placed in the **interrupt wait queue**.
4. If the process time slice expires or an interrupt occurs, the process is removed from the CPU and returned to the **ready queue**.

After the waiting event is completed, the process again returns to the ready queue for CPU allocation.

This cycle continues until the process finishes execution. Finally, the operating system removes the process from all queues and deallocates its PCB and resources.

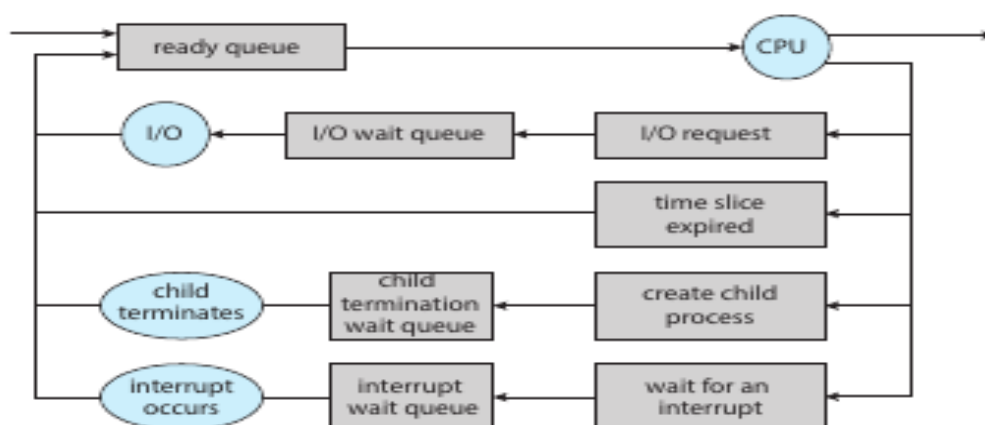


Figure 3.5 Queuing-diagram representation of process scheduling.

9. Process Creation in UNIX Operating System

In UNIX operating systems, a new process is created using the `fork()` system call.

The process that creates another process is called the **parent process**, and the newly created process is called the **child process**.

After `fork()` is executed:

- The child process becomes a duplicate of the parent process.
- Both parent and child continue execution from the instruction after `fork()`.
- The return value of `fork()` is:
 - 0 for the child process
 - Child PID for the parent process

Usually, the child process uses the `exec()` system call to load a new program into memory.

The parent process may use `wait()` to wait until the child process finishes execution.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    pid_t pid;

    pid = fork();

    if(pid < 0) {
        printf("Fork Failed\n");
    }

    else if(pid == 0) {
        printf("Child Process\n");

        execlp("/bin/ls", "ls", NULL);
    }
    else {
        wait(NULL);

        printf("Parent Process\n");
    }
}
```

1. `fork()` creates a child process.
2. The child process executes `execlp()` to load the `ls` command.
3. The parent process waits for the child using `wait()`.
4. After the child finishes, the parent resumes execution.

10. Example where Ordinary Pipes are More Suitable

Ordinary pipes are suitable for simple communication between a parent process and a child process on the same machine.

Easy Example

Suppose a parent process sends file data to a child process.

- Parent process writes data into the pipe.
- Child process reads the data and counts the number of characters.

This is simple one-way communication, so ordinary pipes are suitable.

Example where Named Pipes are More Suitable

Named pipes are suitable when multiple unrelated processes need communication.

Easy Example

Suppose several processes want to send messages to a log server.

- Different processes write messages to the named pipe.
- The server reads the messages and stores them in a log file.

Named pipes are better because:

- No parent–child relationship is needed.
- Multiple processes can communicate through the same pipe.
- Communication may continue even after one process ends.

-
- **Ordinary Pipe** → Best for simple parent–child communication.
 - **Named Pipe** → Best for communication among multiple unrelated processes.

11. Process Creation and Termination in Operating System

A process can create new processes during execution.

The creating process is called the **parent process**, and the newly created process is called the **child process**.

Each process is identified by a unique **Process ID (PID)**. In UNIX/Linux systems, a new process is created using the `fork()` system call.

Steps of Process Creation

1. The parent process calls `fork()`.
2. A child process is created.
3. The child becomes a copy of the parent process.
4. Both parent and child execute concurrently.
5. The child process may use `exec()` to load a new program.
6. The parent may use `wait()` to wait for the child to finish.

A process terminates when it finishes execution and calls the `exit()` system call.

Example:

```
exit(0);
```

After termination:

- Memory is released
- Open files are closed
- I/O resources are deallocated

A parent process can collect the child's exit status using `wait()`.

Zombie Process

A process that has finished execution but still remains in the process table because the parent has not called `wait()` is called a **zombie process**.

Orphan Process

If a parent process terminates before its child, the child becomes an **orphan process**.

In UNIX/Linux, the `init` or `systemd` process becomes the new parent.